



Sistema de Gestión de Contactos

Proyecto Python con Programación Orientada a Objetos

Módulo 2 - Evaluación ABP

Enero 2026

Desarrollado por: José Tomás Latorre

Institución: Sense

[< Anterior](#)

[Siguiete >](#)



Índice de Contenidos

- ✓ Descripción del Proyecto
- ✓ Objetivos y Requerimientos
- ✓ Arquitectura del Sistema
- ✓ Funcionalidades Principales
- ✓ Tecnologías Utilizadas
- ✓ Programación Orientada a Objetos
- ✓ Demostración del Sistema
- ✓ Resultados y Estadísticas
- ✓ Conclusiones

[< Anterior](#)

[Siguiete >](#)



Descripción del Proyecto

Sistema de gestión de contactos desarrollado en Python para organizar y almacenar información personal de clientes.

Características Principales

- ✓ Interfaz de usuario intuitiva por consola
- ✓ Almacenamiento persistente en formato JSON
- ✓ Validación automática de datos
- ✓ Gestión completa CRUD (Create, Read, Update, Delete)

[< Anterior](#)[Siguiete >](#)

Objetivos y Requerimientos

Requerimientos Generales

- ✓ Registro de contactos
- ✓ Edición de contactos
- ✓ Eliminación de contactos
- ✓ Búsqueda por nombre
- ✓ Búsqueda por teléfono

Requerimientos Técnicos

- ✓ Listas y diccionarios
- ✓ POO y encapsulación
- ✓ Validación de datos
- ✓ Pruebas unitarias
- ✓ Documentación completa

[< Anterior](#)[Siguiete >](#)



Arquitectura del Sistema



Clase Contacto

Representa un contacto individual con atributos privados y validación.



Clase GestorContactos

Gestiona la colección de contactos y la persistencia en JSON.



Clase InterfazUsuario

Maneja la interacción con el usuario a través de menús.



Persistencia

Guardado automático en archivo JSON con codificación UTF-8.

[< Anterior](#)[Siguiente >](#)

Funcionalidades Principales

- ✓ Agregar nuevos contactos
- ✓ Editar contactos existentes
- ✓ Eliminar contactos con confirmación
- ✓ Buscar por nombre (parcial)
- ✓ Buscar por teléfono (exacto)
- ✓ Listar todos ordenados alfabéticamente
- ✓ Ver estadísticas del sistema
- ✓ Validación de correo electrónico
- ✓ Prevención de duplicados
- ✓ Manejo de errores robusto

[< Anterior](#)[Siguiente >](#)

Tecnologías Utilizadas

Python 3.8+

JSON

unittest

typing

re (regex)

VS Code

Todas las librerías son parte de la biblioteca estándar de Python

No requiere instalación de dependencias externas

◀ Anterior

Siguiente ▶

Programación Orientada a Objetos

Encapsulación Implementada

```
class Contacto: def __init__(self, nombre, telefono, correo, direccion): self._nombre = nombre #
Atributo privado self._telefono = telefono # Atributo privado @property def nombre(self) -> str:
return self._nombre @nombre.setter def nombre(self, valor: str): if not valor.strip(): raise
ValueError("El nombre no puede estar vacío") self._nombre = valor.strip()
```

◀ Anterior

Siguiente ▶



Resultados y Estadísticas

Código Desarrollado

- ✓ ~500 líneas de código
- ✓ 3 clases principales
- ✓ 25+ métodos implementados
- ✓ Type hints en todo el código
- ✓ Documentación completa

Desafíos Superados

- ✓ Validación de correo con regex
- ✓ Persistencia de datos en JSON
- ✓ Búsqueda case-insensitive
- ✓ Manejo de excepciones
- ✓ Testing comprehensivo

[< Anterior](#)[Siguiente >](#)

Conclusiones

Logros Alcanzados

- ✓ Sistema funcional completo
- ✓ Todos los requerimientos cumplidos
- ✓ Pruebas unitarias exhaustivas
- ✓ Código bien documentado y profesional

[< Anterior](#)[Siguiente >](#)